KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Library Digital Catalog and Membership Management System

Submitted by

Yuvani Dharan T

25BIT064

Under the Guidance of

DHILIBAN

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: 2026–2027

October 2026

DECLARATION

I hereby declare that the project entitled “Library Digital Catalog and Membership Management System” submitted in partial fulfillment of the requirements for the course Java Programming is a record of my work carried out for academic purposes.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title at this or any other institution.

I have acknowledged the sources of information used in this project wherever applicable.

Date: October 2026

Place: Coimbatore

Signature: Yuvani Dharan T

CERTIFICATE

This is to certify that the project entitled “Library Digital Catalog and Membership Management System” is a bona fide project work carried out by Yuvani Dharan T (25BIT064), B.Sc. Information Technology, KG College of Arts and Science, as part of the Java Programming course.

The project has been prepared to demonstrate the application of Java programming concepts, including classes, objects, methods, collections, loops, conditional statements, and graphical user interface components.

Place: Coimbatore

Date: _____

Guide / Technical Trainer: DHILIBAN _____

Head of the Department: _____

TABLE OF CONTENTS

S. No.	Topics	Page No.
1	Introduction	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	Program Analysis	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scope and Limitations	
3	System Design	
3.1	Modules	
4	Methodology	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	Source Code	
6	Input and Output Screens	
7	Discussion and Conclusion	
7.1	Discussion	
7.2	Conclusion	
8	References	

ABSTRACT

The Library Digital Catalog and Membership Management System is a Java-based application designed to maintain a simple list of library members and books. The program provides options to add and view members, add and view books, issue books, and return books. It uses Java classes to represent books and members, ArrayList collections to store records during execution, and JOptionPane dialogs to interact with the user.

The system keeps track of whether each book is issued or available. When a book is issued, the program checks its ID and current status before changing the status. Similarly, a return operation changes an issued book back to available. The project demonstrates object-oriented programming and basic record-management operations. Records are held in memory and are not saved permanently after the application closes.

1. INTRODUCTION

Libraries manage books and members and need a clear way to check which books are available or issued. When these details are maintained manually, searching for records and tracking book status can take additional time. A simple computer-based system can make these basic operations easier to perform.

The Library Digital Catalog and Membership Management System is developed using Java. It presents a menu through JOptionPane and allows the user to add and display member records, add and display books, issue a book, return a book, or exit the application.

1.1 Objective

- To create a simple Java application for maintaining member and book details.
- To allow users to add and view library members.
- To allow users to add and view books with their availability status.
- To implement book issue and return operations using a book ID.
- To demonstrate classes, objects, ArrayList, methods, loops, conditional statements, and JOptionPane.

1.2 Overview

The application displays a seven-option menu. Member and book records are stored in ArrayList objects while the program is running. Each book has an ID, title, and issued status. Each member has an ID and name. The issue and return functions search for a matching book ID and update its status when the requested operation is valid.

1.3 Features

- Add Member: records a member ID and name.
- View Members: displays all registered members or a message if none exist.
- Add Book: records a book ID and title.
- View Books: displays each book and whether it is issued or available.
- Issue Book: marks an available book as issued.
- Return Book: marks an issued book as available.
- Exit: closes the application.

2. PROGRAM ANALYSIS

Program analysis identifies the problem, required operations, and data needed for the application. The system focuses on basic book and member records and the current issue status of each book.

2.1 Problem Definition

Maintaining book and member details manually can make it difficult to find a particular record or quickly check whether a book is available. The project provides a simple menu-based program to organize these details and handle basic issue and return operations.

2.2 Project Overview

The application is written in Java and uses the Swing JOptionPane class for dialog boxes. The Member class stores an integer ID and a name. The Book class stores an integer ID, a title, and a boolean issued value. Two ArrayList collections store the member and book objects during execution.

2.3 Scope and Limitations

Scope

- Maintain member details during the current session.
- Maintain book details and issue status during the current session.
- Search for books by ID when issuing or returning them.
- Display records using dialog boxes.

Limitations

- Records are stored in memory and are lost when the application closes.
- The current program does not use a database or file storage.
- The program does not prevent duplicate member IDs or book IDs.
- Input validation for non-numeric IDs and cancelled input can be improved.
- The system does not maintain due dates, fines, or issue history.

3. SYSTEM DESIGN

System design defines the data structures and functions used by the program. The application uses two model classes and a main class containing the menu and operations.

3.1 Modules

1. Add Member Module

Accepts a member ID and member name, creates a Member object, adds it to the members list, and displays a confirmation message.

2. View Members Module

Checks whether the members list is empty. If it is empty, the application displays “No members found.” Otherwise, it combines the member records into a string and displays them.

3. Add Book Module

Accepts a book ID and title, creates a Book object with issued set to false, adds it to the books list, and displays a confirmation message.

4. View Books Module

Displays all stored books. The Book toString method shows the ID, title, and status as either “Issued” or “Available.”

5. Issue Book Module

Accepts a book ID and searches the books list. If the book exists and is not issued, its issued field is set to true. If it is already issued, a message is shown. If no matching ID exists, the application reports that the book was not found.

6. Return Book Module

Accepts a book ID and searches the books list. If the book exists and is issued, its issued field is set to false. Otherwise, the application reports that the book was not issued or was not found.

7. Exit Module

The exit option terminates the application. Closing the menu dialog also ends the menu loop.

4. METHODOLOGY

The project follows a basic software development process: identify the requirements, design the classes and modules, implement the program, test each menu option, and document the outcome.

4.1 Problem Analysis

The requirements were identified as maintaining member details, maintaining book details, displaying records, and updating book status through issue and return operations.

4.2 System Design

The Member and Book classes represent the main records. The LibrarySystem class contains static ArrayList collections and methods for each menu operation. A while loop repeatedly displays the menu, and a switch statement routes the selected option to the corresponding method.

4.3 Program Development

1. Import java.util.* for ArrayList and related utilities.
2. Import javax.swing.* to use JOptionPane dialog boxes.
3. Create Member and Book classes with constructors and toString methods.
4. Create lists to hold member and book objects.
5. Implement add, view, issue, and return methods.
6. Use a menu loop and switch statement to control the application.

4.4 Testing and Validation

Test Case	Action	Expected Result
Add member	Enter ID and name	Member added confirmation
View members	Select View Members	Stored member records are displayed
Add book	Enter book ID and title	Book added confirmation
View books	Select View Books	Book status is displayed
Issue available book	Enter an available book ID	Book status changes to Issued
Issue issued book	Enter the same issued book ID	Already Issued message
Return issued book	Enter an issued book ID	Book status changes to Available
Unknown book ID	Enter an ID not in the list	Book not found message
Empty list	View books or members before adding records	Empty-list message

4.5 Result and Documentation

The application is designed to perform the specified member and book management operations through dialog boxes. The expected results include adding records, viewing stored data, and updating book availability during issue and return operations. Actual screenshots can be inserted in Section 6 after running the program.

4.6 Algorithm

7. Start the program.
8. Initialize the member and book lists.

9. Display the menu using JOptionPane.
10. Read the selected menu option.
11. Call the corresponding add, view, issue, or return method.
12. For issue or return, search the book list using the entered book ID.
13. Display the appropriate result message.
14. Repeat the menu until the user exits or closes the dialog.
15. End the program.

4.7 Flowchart

Start → Initialize lists → Display menu → Read choice → Execute selected operation → Display result → Return to menu → Exit selected? If yes, end; otherwise, display the menu again.

5. SOURCE CODE

```
import java.util.*;
import javax.swing.*;

class Member {
    int id;
    String name;

    Member(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public String toString() {
        return id + " - " + name;
    }
}

class Book {
    int id;
    String title;
    boolean issued;

    Book(int id, String title) {
        this.id = id;
        this.title = title;
        this.issued = false;
    }

    public String toString() {
        return id + " - " + title + (issued ? " (Issued)" : " (Available)");
    }
}

public class LibrarySystem {
    static ArrayList<Member> members = new ArrayList<>();
    static ArrayList<Book> books = new ArrayList<>();

    public static void main(String[] args) {
        while (true) {
            String menu = JOptionPane.showInputDialog(
                "Library System\\n\\n" +
                "1. Add Member\\n" +
                "2. View Members\\n" +
                "3. Add Book\\n" +
                "4. View Books\\n" +
                "5. Issue Book\\n" +
                "6. Return Book\\n" +
                "7. Exit\\n\\n" +
                "Enter choice:"
            );

            if (menu == null) break;
            int choice = Integer.parseInt(menu);

            switch (choice) {
                case 1: addMember(); break;
                case 2: viewMembers(); break;
                case 3: addBook(); break;
                case 4: viewBooks(); break;
```

```
            case 5: issueBook(); break;
            case 6: returnBook(); break;
            case 7: System.exit(0);
        }
    }
}

static void addMember() {
    int id = Integer.parseInt(JOptionPane.showInputDialog("Enter Member ID:"));
    String name = JOptionPane.showInputDialog("Enter Member Name:");
    members.add(new Member(id, name));
    JOptionPane.showMessageDialog(null, "Member Added!");
}

static void viewMembers() {
    if (members.isEmpty()) {
        JOptionPane.showMessageDialog(null, "No members found.");
        return;
    }
    String data = "";
    for (Member m : members) data += m + "\\n";
    JOptionPane.showMessageDialog(null, data);
}

static void addBook() {
    int id = Integer.parseInt(JOptionPane.showInputDialog("Enter Book ID:"));
    String title = JOptionPane.showInputDialog("Enter Book Title:");
    books.add(new Book(id, title));
    JOptionPane.showMessageDialog(null, "Book Added!");
}

static void viewBooks() {
    if (books.isEmpty()) {
        JOptionPane.showMessageDialog(null, "No books available.");
        return;
    }
    String data = "";
    for (Book b : books) data += b + "\\n";
    JOptionPane.showMessageDialog(null, data);
}

static void issueBook() {
    int bookId = Integer.parseInt(JOptionPane.showInputDialog("Enter Book ID to Issue:"));
    for (Book b : books) {
        if (b.id == bookId) {
            if (!b.issued) {
                b.issued = true;
                JOptionPane.showMessageDialog(null, "Book Issued!");
            } else {
                JOptionPane.showMessageDialog(null, "Already Issued!");
            }
            return;
        }
    }
    JOptionPane.showMessageDialog(null, "Book not found!");
}

static void returnBook() {
    int bookId = Integer.parseInt(JOptionPane.showInputDialog("Enter Book ID to Return:"));
    for (Book b : books) {
        if (b.id == bookId) {
            if (b.issued) {
                b.issued = false;
                JOptionPane.showMessageDialog(null, "Book Returned!");
```

```
            } else {
                JOptionPane.showMessageDialog(null, "Book was not issued!");
            }
            return;
        }
    }
    JOptionPane.showMessageDialog(null, "Book not found!");
  }
}
```

6. INPUT AND OUTPUT SCREENS

Insert screenshots captured from the running LibrarySystem application in this section. The suggested screenshots are:

- Main menu showing all seven options.
- Add Member dialog and member-added confirmation.
- View Members output.
- Add Book dialog and book-added confirmation.
- View Books output showing Available status.
- Issue Book result showing Issued status.
- Return Book result showing Available status.

\nNote: Screenshots should be captured from the actual program execution and added before final submission.

7. DISCUSSION AND CONCLUSION

7.1 Discussion

The Library Digital Catalog and Membership Management System demonstrates how Java can be used to manage simple records. The Member and Book classes group related data, while ArrayList stores multiple objects. The menu-driven interface uses JOptionPane to collect input and display messages. Conditional statements ensure that a book is not issued twice and that only an issued book can be returned.

The current implementation stores data only for the duration of the program. It also expects valid numeric input for IDs and menu choices. Database storage, duplicate-ID checks, and improved input validation could be added in a future version.

7.2 Conclusion

The project implements the main functions of a basic library catalog and membership system: adding and viewing members, adding and viewing books, issuing books, and returning books. It provides practical experience with Java classes, objects, ArrayList, methods, loops, conditional statements, and Swing dialog boxes. Future enhancements may include permanent database storage, search by title, due dates, fine calculation, and a more complete graphical interface.

8. REFERENCES

- Herbert Schildt, Java: The Complete Reference.
- Oracle Java Documentation: https://docs.oracle.com/en/java/
- Java Swing Documentation: https://docs.oracle.com/javase/tutorial/uiswing/